

[HIDDEN]-ERE

Annotation Guideline 1.0

Abstract

This guideline is the guideline for our joint machine learning (ML) model and dataset mention annotation task on CS publications. We provide term definitions of ten entities related to ML models or datasets and should guide the annotators to understand the task and do the annotation work. Considering the difficulty of the annotation task, we also provide examples from annotated publications, accompanying the term definitions. We would also like to emphasize that since this is the first version, the annotator should first read the whole guideline for understanding our task, and report problems when annotating the trial publication(s) back to us when using the guideline..

Resources

This guideline is inspired by,

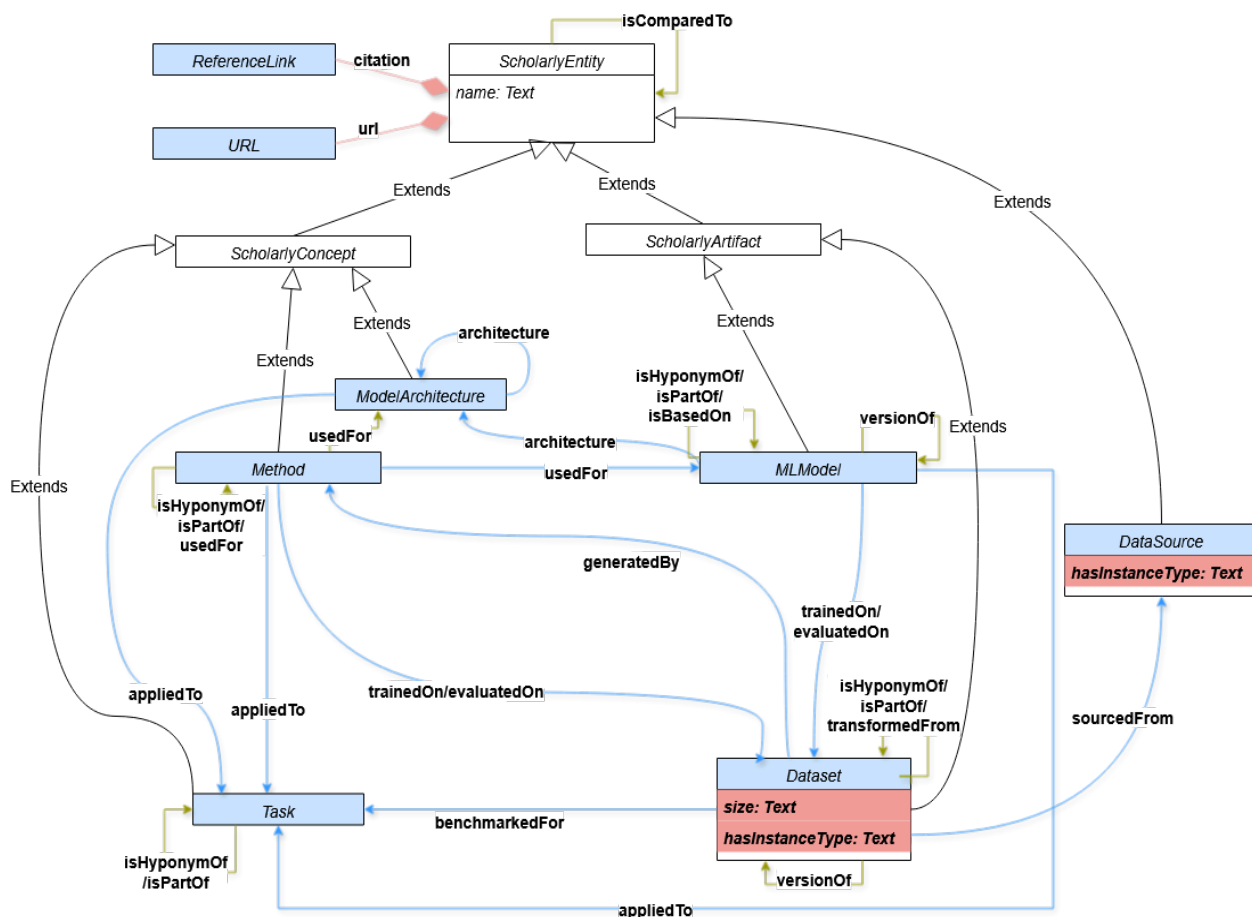
- SciERC annotation guideline
http://nlp.cs.washington.edu/sciE/annotation_guideline.pdf
- ACL TEC annotation guideline
<https://doi.org/10.13140/RG.2.1.1939.1446>

Annotation Tool

INCEpTION 20.4.

Data model

We provide a data model for the joint annotation task. This data model should facilitate the annotators to better understand each term defined in the following sections.



The annotation tagset (the terms defined in the [Entity Category](#) and [Relation Category](#)) described in this document can be mapped to colored (red, blue and yellow green) entities, attributes and relations in the above class diagram with a few exceptions which are [generic mentions](#).

Preliminary remarks

Generics and coreferences.

Generic mentions refer to string spans that cannot be identified as one of the name-based entities [listed below](#). However, they are interesting in that they refer to *informal* mentions of ML models and datasets, and thus, should be captured. "Our dataset" (class1), "the model" (class1), "30 different existing computer vision datasets" (class2), "pretrained multilingual language models" (class2) and "both methods" (class3) are examples of generic mentions.

Some of the generic mentions, like "our dataset" or "the model", can have coreferences to name-based entities that have been annotated previously in the text. If possible, they should be linked to the last or next mention, and should be related as such using the "coreference" relation.

Determiners and articles.

See also ACL RD TEC Guideline, 3.3 "Determiners".

In general, annotators are instructed to not include articles like "a", "the". However, articles should be considered for *generic mentions only*. For example, the determiner "a" in the string span "a large-scale, diverse dataset" should be included. In some cases, the article needs to be included to guard the semantical meaning of the entity of interest.

Abbreviations.

See ACL RD TEC Guideline, 3.3 "Abbreviations".

Term-abbreviation sequence.

See ACL RD TEC Guideline, 3.3 "Term-abbreviation sequence".

Term broken by abbreviations.

See ACL RD TEC Guideline, 3.3 "Term broken by abbreviations".

Terms broken by modifiers.

As a general rule, if noun-based string spans are broken by one or more modifiers, annotate the whole string span. For example, in the string span "a large-scale, diverse dataset", "diverse" and "large-scale" are modifiers, and thus, the whole string span should be annotated.

Generic nouns.

Terms can be accompanied by generic nouns, e.g., the term "dataset" in the string span "the SQuAD v1.1 dataset" or "a large-scale, diverse dataset". We will consider these generic nouns for *generic mentions only* and not for named entities. Thus, "dataset" should not be included in the first example, but the second instead.

Proper nouns.

Proper nouns (names) are string spans corresponding to an explicit name-based mention in the text that relates to one entity of interest (see [Entity Category](#)). Examples are "Social Bias Inference Corpus (SBIC)", "BERT", "SQuAD", "GLUE". As stated earlier, [determiners and articles](#) should not be included during annotation, neither should [generic nouns](#).

Adjectival modifiers.

See also ACL RD TEC Guideline, 3.3 "Adjectival modifiers". In many cases, entities are modified by adjectives, for example, in the strings "well-known CommonCrawl" or "multilingual BERT (mBERT)". In these cases you should check whether the meaning is changed when the adjectives are removed. If yes, do not remove the adjective. For example, "multilingual" is an important modifier in the second example, whereas "well-known" is not important. Adjectival modifiers should be included for generic mentions, e.g. "improved QA-pair models".

Conjunctions, prepositions, and "constructed entities".

See ACL RD TEC Guideline, 3.3 "Conjunctions and prepositions".

Plurals.

We do not consider plural mentions like “the image datasets” as noun-based entities, in particular instances of a model or dataset. Instead, we treat them as generic mentions.

Scope of relations.

We limit the scope of relations to one paragraph.

Excluded from annotation.

Annotators are instructed to *not include* the “References” section. When annotators detect non-sentence format of paragraphs (e.x. Extraction of a table from PDF), then annotators should annotate the whole paragraph as Corrupted.

Other general remarks.

- Keep the annotations flat, do not stack much.
- Watch out for judgemental statements in descriptions.

Term Definitions and Examples

Entity Category

In this section, we provide the definition and corresponding annotation example for each entity. We note that each entity name is in font **Bold**, and the definition or description of the entity follows the entity names. Examples of annotation are screenshots from Inception tool.

MLModel

This entity type refers to a string span that represents a name based entity of a machine learning model. A MLModel usually is based on some machine learning (ML) architecture, and can be applied to some ML tasks. For neural network based machine learning models, such a string span should correspond to the executable resource of the model. We give an example in Figure 1.

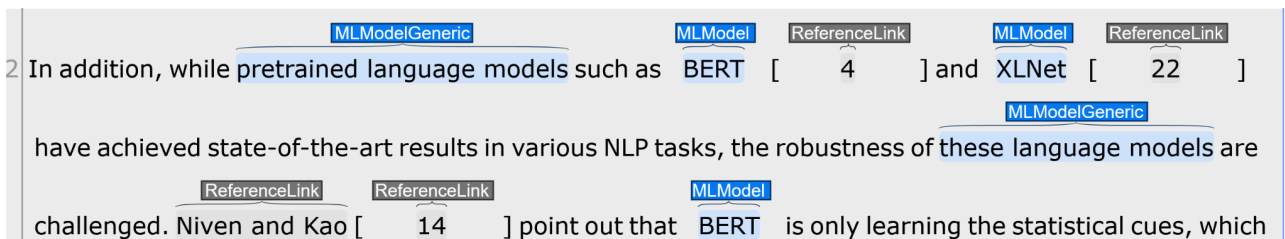


Figure 1. In this case, “BERT”, “XLNet” are MLModel instances.

MLModelGeneric

This entity type refers to a string span corresponding to an MLModel in the text, which cannot be identified as a named entity. Typical examples are: “our model”, “the model” or plural mentions like “conventional models”, or “QA-pair retrievers”. For MLModelGeneric. Examples are given in Figure 2 & 3.

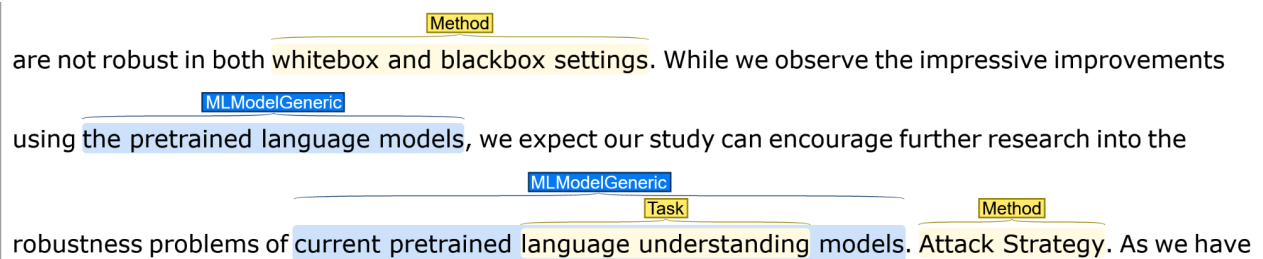


Figure 2. In this example, both “the pretrained language models” and “current pretrained language understanding models” are informal mentions of a group of machine learning models, thus are categorized as MLModelGeneric.

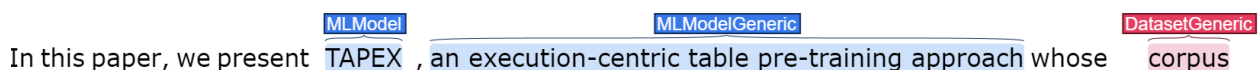
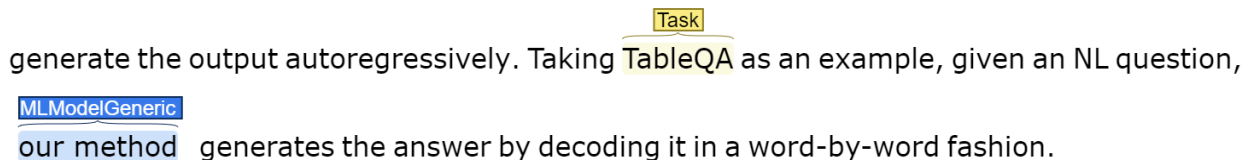


Figure 3. We give two more examples of MLModelGeneric. “our method” is an anonymous mention to the machine learning model proposed by the authors, thus is categorized as MLModelGeneric. Note: this instance does not contain other information apart from the possessional relation between the referred model and the paper. “an execution-centric table pre-training approach” is also an anonymous mention of the model TAPEX, but it provides certain information regarding the referred model.

ModelArchitecture

This entity type refers to a name based string span that mentions a ML model architecture or conceptual type. We clarify the difference between ModelArchitecture and other model related entity types as below:

- The difference between Method and ModelArchitecture is that a Method may contain other algorithmic steps and multiple model architectures.
- Certain name based spans can be assigned to either MLModel or ModelArchitecture according to the context. We assign such a span to ModelArchitecture when the semantics refer to the conceptual structure, or more specifically this structure is being used or based on or modified in the context.
- Between MLModelGeneric and ModelArchitecture, when the MLModelGeneric refers to a general type of some machine learning models, the string span should be double annotated as ModelArchitecture and MLModelArchitecture.

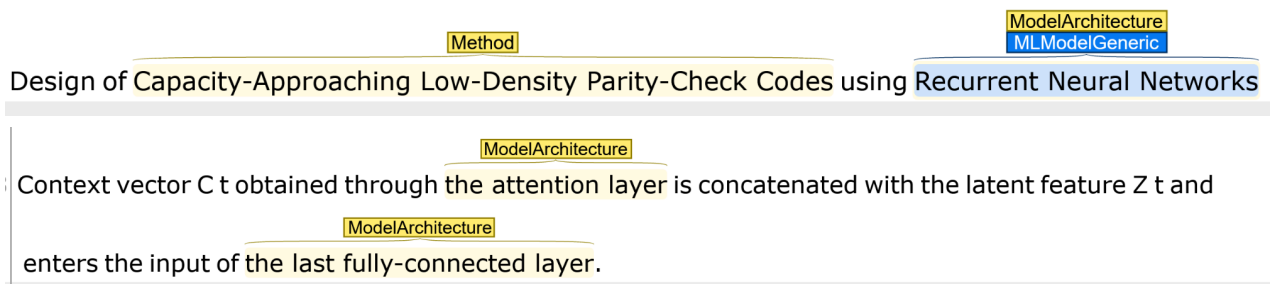


Figure 4. We give examples for ModelArchitecture in above screenshots.

Method

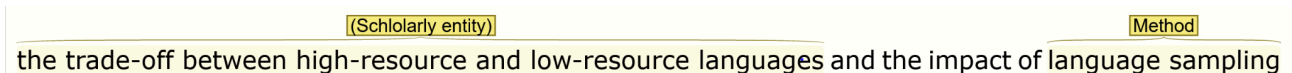
This entity type refers to a string span that refers to a *method*. Methods could be

- noun-based phrases from which a ModelArchitecture can be *derived*;
- *results* from a conceptualization, compared to an executable model with determined parameter values;
- things that are *produced by* a model, e.g. "GloVe embeddings", "ELMo embeddings", "word embeddings", "sentence embeddings";
- *components* of a model architecture that cannot be categorized as ModelArchitecture, e.g. "loss function", "Adam optimizer".

Methods spans should be considered as generics, and hence all rules apply.

Note

1. Machine learning representations should be annotated as Method entities.
2. We should annotate "unsupervised learning", "supervised learning" etc. as Methods.
3. In the case of "GloVe embeddings", "ELMo embeddings", "GloVe" and "ELMo" should be stack annotated as ModelArchitecture.



Dataset

This entity type refers to a string span corresponding to an explicit noun-based dataset object in the text (e.g. ‘Social Bias Inference Corpus’, ‘SBIC’, ‘SQuAD’, ‘GLUE’) . If the string span includes both the full-length name and the acronym, we include them both separately as a Dataset (see [Preliminary Remarks “Abbreviations”](#)). An example is given in Figure 5.

Note A Dataset can contain a set of other Dataset instances, as in the case of “CommonCrawls” datasets. And the relation “isPartOf” can be used for these Dataset instances for this purpose.

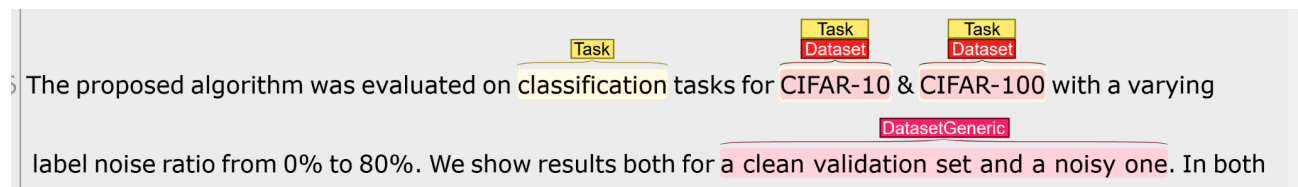


Figure 5. Examples of Dataset spans.

DatasetGeneric

This entity type refers to a string span corresponding to a mention referring to one or more concrete named entity based datasets in the text, but is not an explicit name-based dataset.

Note Data source name (e.g. Wikipedia) can be included in a DatasetGeneric mention (e.g. “the Wikipedia dataset”, “the Wikipedia datasets”) .

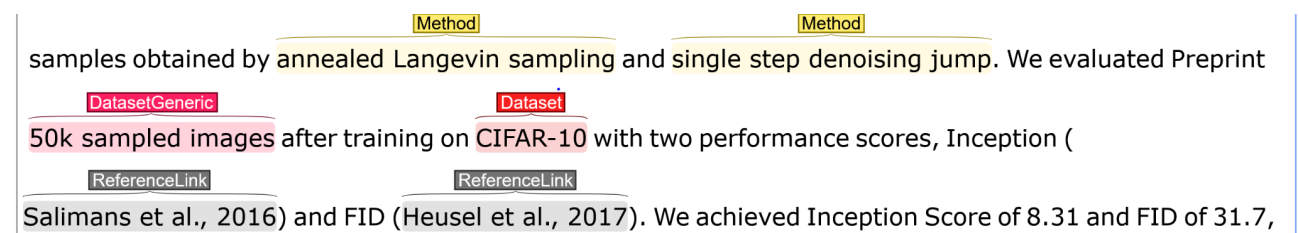


Figure 6. “50k sampled images” is not a name-based dataset mention, but provides additional type and size information in the mention.

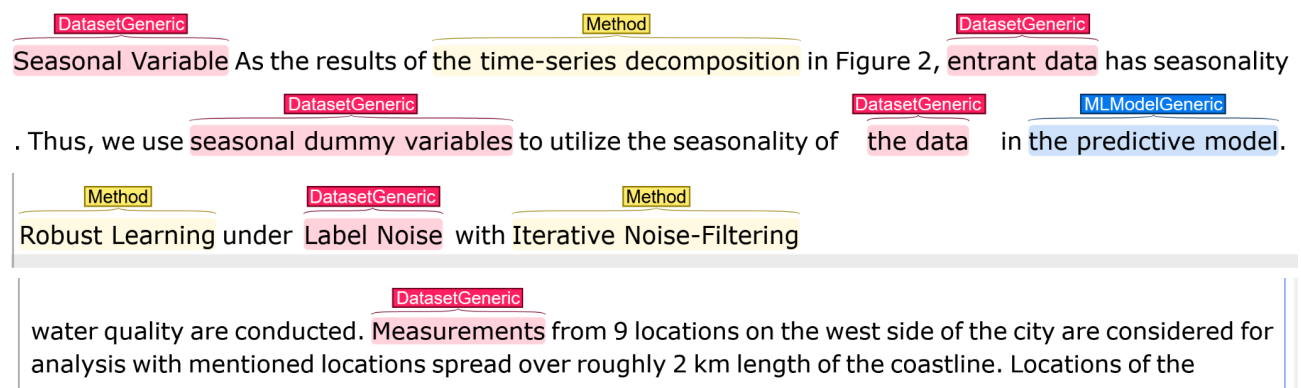


Figure 7. More DatasetGeneric examples.

DataSource

This entity type refers to a string span of *data source information*, e.g. “Wikipedia”, “Twitter” and other possible explicit web or unstable, or not static sources. The difference to a Dataset is that the DataSource has unstable reproducibility.

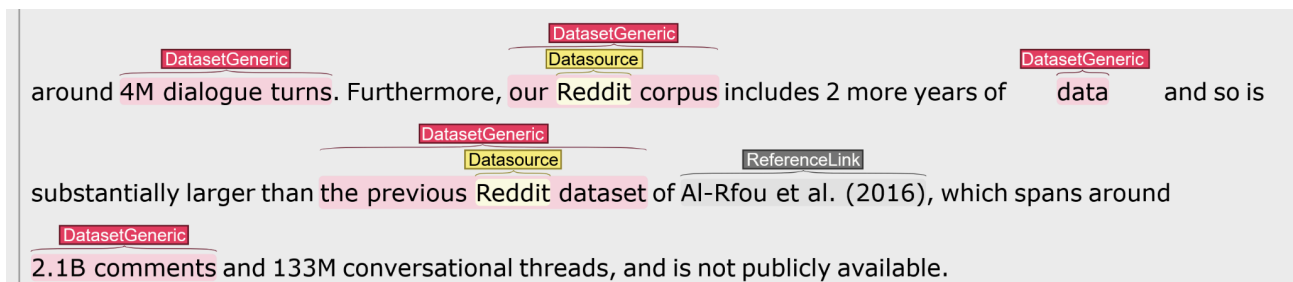


Figure 8. “Reddit” is a DataSource span.

ReferenceLink

A ReferenceLink instance refers to a string span that represents a reference in the text. A ReferenceLink may present in different style, but it requires to be linkable to the bibliography section at the end of the paper.

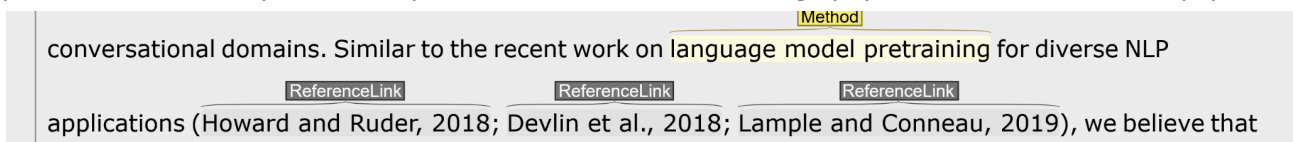


Figure 9. Examples of ReferenceLink.

Task

This entity type refers to a string span which represents a specific ML task or a collection of themed ML tasks.

Note We only annotate a name-based string as a Task, and omit descriptive processes for a task. A counterexample is “removing the next sentence prediction objective”.

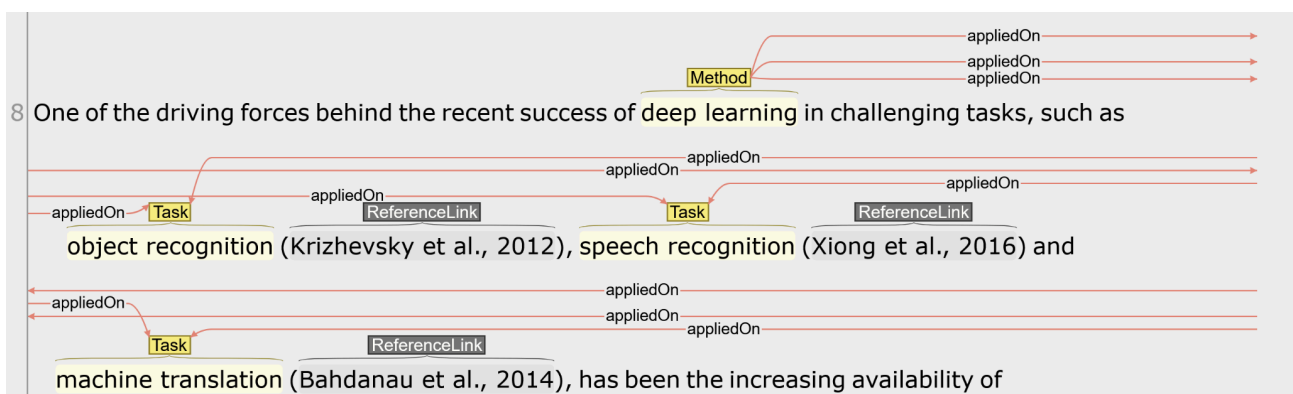


Figure 10. Examples of Task.

URL

An URL instance is a string span that is a url in the text.

Relation Category

Relations should be preferably annotated within one sentence, given the entities of the relation appearing in the same sentence, then by descending order the local context (one sentence before and one sentence after) and the larger context scale.

Only one relation is allowed between two identical entity annotations.

One entity relation can have multiple relations to different entity annotations.

Finally, the semantics of each relation should capture either a general rule of between the annotated entity spans, or a specific existing action or relationship between the concrete entities represented by the entity span; this exclude the future application, potential relationship or other non-factual relationship among entities.

appliedTo

This relation type describes a relation between a machine model related scholarly artifact A (any instance from MLModel, MLModelGeneric, Method, ModelArchitecture) and a Task entity, where the artifact A has been applied on the task in the context.

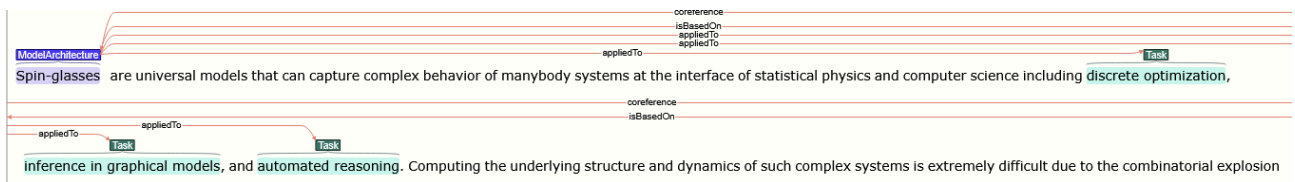


Figure 11. Examples of *appliedTo* relation on INception tool.

architecture

This relation type define a relation in one of the two cases: (1) between a MLModel/MLModelGeneric entity A and a ModelArchitecture entity B, where A is backboned by entity B, regardless of B being the whole architecture or architectural component (2) between a ModelArchitecture C and a ModelArchitecture D, where D should be an architectural component of C.

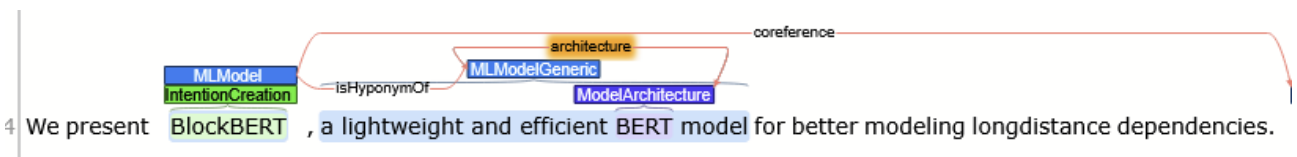


Figure 12. Example of architecture relation.

benchmarkFor

This relation type defines a relation between a Dataset/DatasetGeneric entity A and a Task entity B, where A has/have been used for the task B, allowing A to be a benchmark dataset/datasets for the task B. Therefore when A is of type DatasetGeneric, A should semantically refer to concrete dataset or datasets (not an unlimited set of datasets or the dataset in general without specificity).

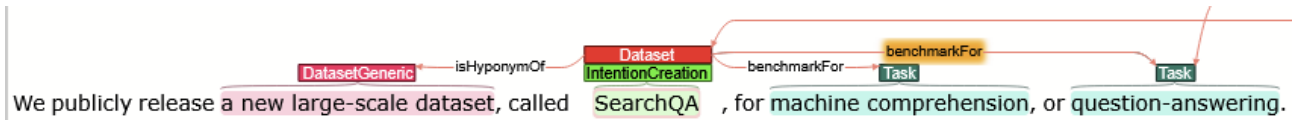


Figure 13. Examples of benchmarkFor relation.

citation

This relation type defines a relation between a scholarly artifact entity and an instance of ReferenceLink, where the former should be linked to the latter.

Notes: We do the citation link in a broadly defined fashion where the ReferenceLink instance should be linked to nearby entities (name entities are prioritized).

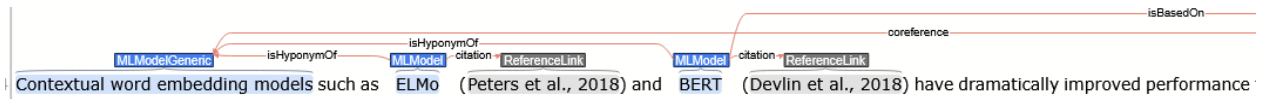


Figure 14. Example of citation relation.

coreference

This relation type represents a coreference relation between *generic entity types* and *concrete scholarly entities* and between each of them (example for Dataset: DatasetGeneric-coreference->Dataset, Dataset -coreference-> Dataset, DatasetGeneric -coreference->DatasetGeneric; same for MLModel ...)

It is used to describe that two annotations refer to the exact *same Entity*.

The coreference relation is preferably annotated to the last mention of the same entity.

The coreference relation is not hierarchical, as opposed to isHyponymOf and isPartOf.

Note: If possible the Generic instances should be linked to the next mention of the co-referenced entity (e.g. Dataset, MLMethod) with the `coreference` relation. For simplicity and limiting by the fact that the whole document cannot be shown at once altogether on INCePTION, we limit annotating coreferences inside the same paragraph.

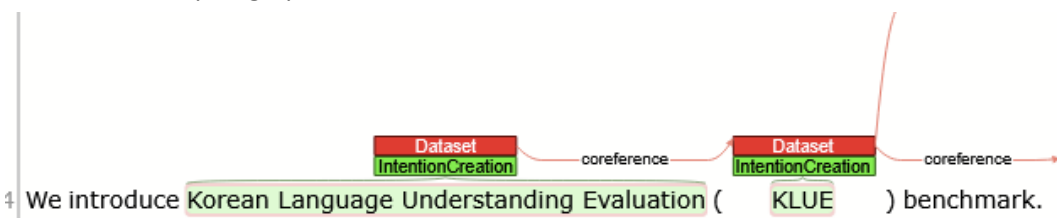


Figure 15. Example of coreference relation.

evaluatedOn

This relation type represents a evaluate-on relation between an instance of MLModel/MLModelGeneric/Method and an instance of Dataset/DatasetGeneric. The indication of evaluation semantics can be model performance, evaluation procedures, model testing etc.

Note: the evaluatedOn relation should capture an evaluation done in general or in a specific case, but does not include a potential future possibility.

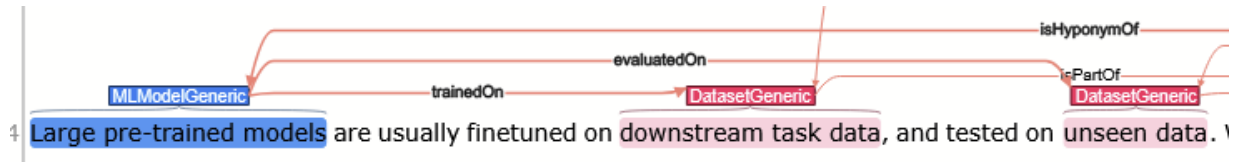


Figure 16. Example of evaluatedOn relation.

generatedBy

This relation type describes a Dataset/DatasetGeneric entity being a product of a Method or MLModel/MLModelGeneric entity.



Figure 17. Example of generatedBy relation.

hasInstanceType

The relation type describes the type/property of a Dataset or DatasetGeneric. The entity span containing the instance type information should be annotated as DatasetGeneric.

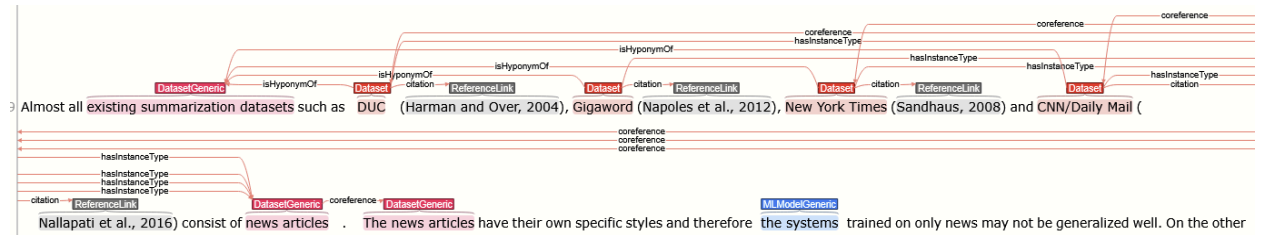


Figure 18. Examples of hasInstanceType relation.

isBasedOn

The relation type describes the transformation or extension among MLModel and MLModelGeneric entities.

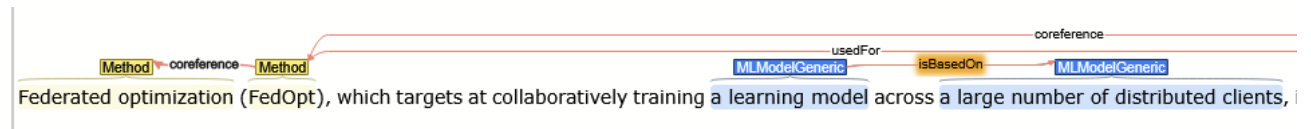


Figure 19. Example of isBasedOn relation.

isComparedTo

This relation type captures identical or compatible entity mentions being compared in the context. The common comparison between entities can be for performance and entity features.

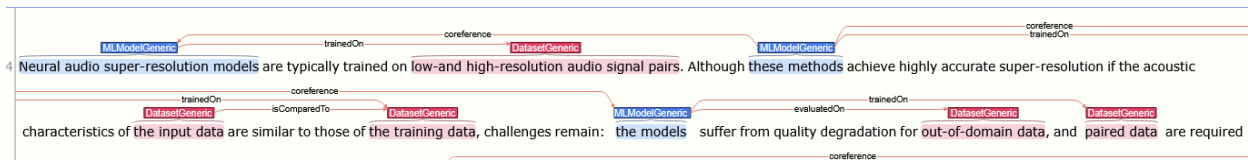


Figure 20. Example of isComparedTo relation.

isHyponymOf

This relation type captures the subclass or instance-class relation between two entity mentions of the identical or compatible entity types.

Note: between coreference and isHyponymOf, when a context allows both annotation, as shown in Figure 21, we prioritize isHyponymOf relation as it can organize more named entities in the hierarchical structure.

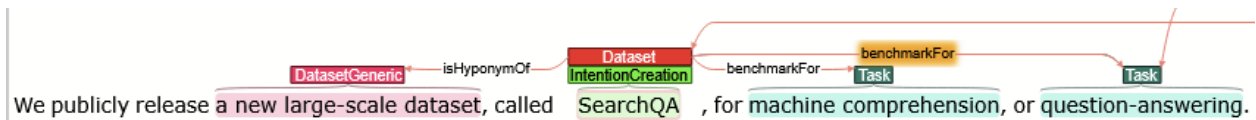


Figure 21. Example of isHyponymOf relation.

isPartOf

An isPartOf relation describes a part-of relation between entities of identical or compatible types among Dataset / DatasetGeneric, Method or MLModel / MLModelGeneric. isPartOf is commonly referring to the dataset splits, collections or lists of datasets or machine learning models or methods.

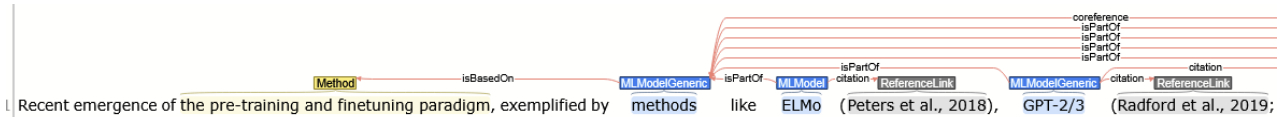


Figure 22. Example of isPartOf relation.

size

This relation captures the size or volume information of a Dataset / DatasetGeneric entity mention in the context. Similar to hasInstanceType relation, the entity span containing the size information should be annotated as DatasetGeneric in the first place.

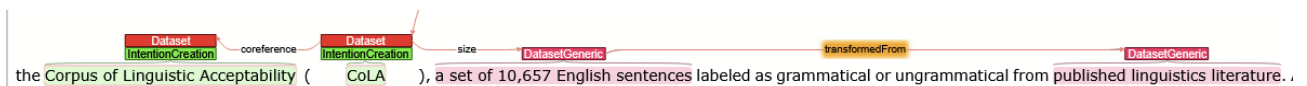


Figure 23. Example of size relation.

sourcedFrom

This relation represents the relation between a Dataset / DatasetGeneric entity mention and a DataSource entity, carrying the original source information.

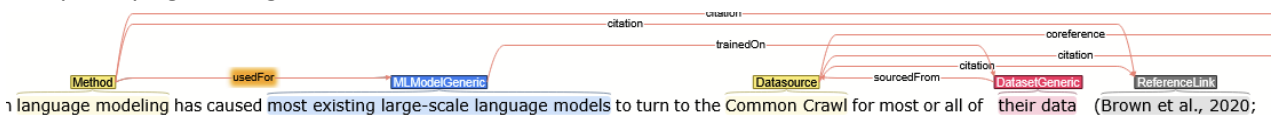


Figure 24. Example of sourcedFrom relation.

trainedOn

This relation type represents a train-on relation between a MLModel / MLModelGeneric / Method and a Dataset / DatasetGeneric.

Note: similar to evaluatedOn relation, the trainedOn relation should capture a training process done in general or in a specific existing case, but does not include a potential future possibility.

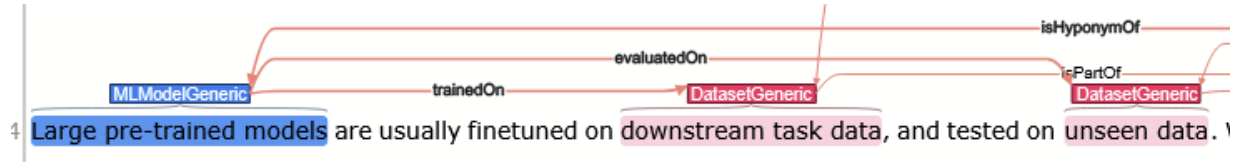


Figure 25. Example of trainedOn relation.

transformedFrom

This relation represents the transformation of different Dataset / DatasetGeneric entities.

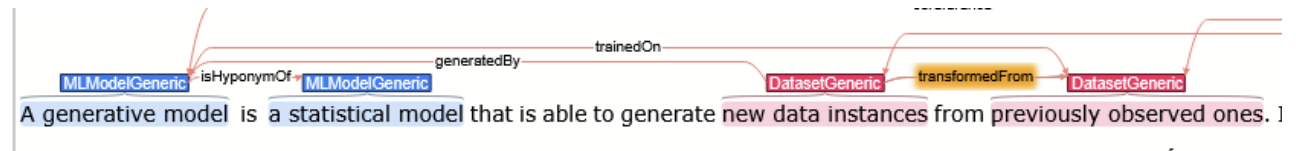


Figure 26. Example of transformedFrom relation.

usedFor

This relation captures a Method entity being used in a MLModel / MLModelGeneric / ModelArchitecture for its functionality.

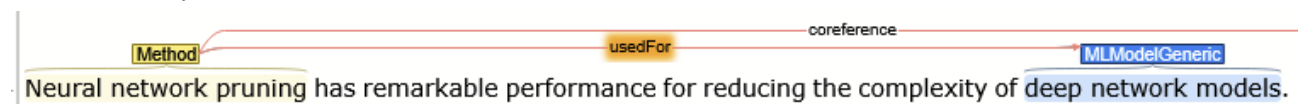


Figure 27. Example of usedFor relation.

url

This relation links the URL entity to another entity, capturing the external resource location information of a named or informal mentioned entity.

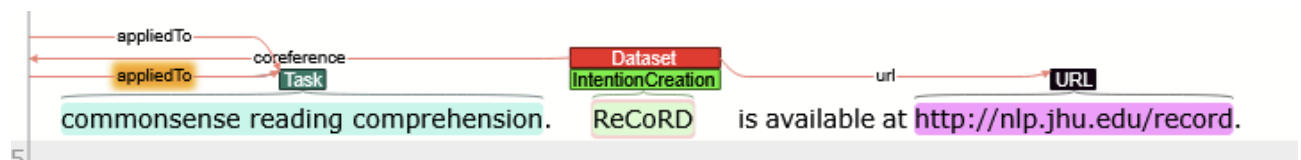


Figure 28. Example of url relation.